

Disciplina:
**Laboratório de
Banco de Dados
Avançado**

Unidade 6:

Triggers

Especificação de restrições gerais.
Introdução às triggers em SQL.



Criando uma exceção personalizada

- Comando:
- `RAISE_APPLICATION_ERROR(error_number, error_message);`
- `error_number`: Um número de erro no intervalo de -20000 a -20999. Estes valores são reservados para mensagens personalizadas.
- `error_message`: Uma string de até 2048 caracteres que descreve o erro.
- Exemplo:
- `RAISE_APPLICATION_ERROR(-20001,Mensagem de erro personalizada');`

Conceitos



- Trigger é um bloco de código PL/SQL armazenado no banco de dados que é disparado automaticamente mediante uma ação.
- O disparo pode ocorrer de duas formas:
 - Alterações feitas em registros de uma determinada tabela do banco de dados;
 - Ações em nível de sistema por um trigger de sistema.

Trigger de Banco de Dados

- Podemos criar um trigger que dispare quando fizermos um update em determinada tabela, e que ocorra após o update dos registros.
- Podemos definir que este trigger disparará para cada linha afetada pelo comando DML, chamado de trigger de linha, ou se executará uma única vez independente de quantas linhas sofrerem alterações.

Trigger de Sistema

- São objetos criados em nível de sistema e não de tabelas.
- Quando falamos em sistemas nos referimos ao banco de dados, mais precisamente a ações que ocorrem nele.
- Seu uso é mais comum pelos DBAs (administradores de banco de dados) e é uma ferramenta poderosa quando utilizada com criatividade.
- Os triggers de sistema disparam conforme um evento que acontece no banco de dados.

Alguns eventos de sistema

- **startup**: quando o banco de dados é aberto;
- **shutdown**: antes de o banco de dados iniciar o shutdown(fechamento). Se for shutdown abort este evento não é disparado;
- **servererror**: quando um erro ocorre.

Alguns eventos de sistema

- **after logon**: depois de uma conexão ser completada no banco de dados;
- **before logoff**: quando o usuário desconecta do banco de dados;
- **before create / after create**: quando um objeto é criado no banco de dados (comando create), exceto o comando create database;
- **before alter / after alter**: quando um objeto é alterado no banco de dados (comando alter), com exceção do comando alter database;
- **before drop / after drop**: quando um objeto é eliminado do banco de dados (comando drop);

Exemplo que registra quando o usuário conecta no banco de dados:

```
1 v create table hist_usuario(  
2     nm_usuario varchar2(50)  
3     ,dt_historico date  
4     ,ds_historico varchar2(4000)  
5 );  
6  
7 v create or replace trigger tgr_hist_conexao_usuario  
8 after logon on database  
9 begin  
10     insert into hist_usuario values  
11         (ORA_LOGIN_USER,sysdate,'Conexão com o banco de dados'||ORA_DATABASE_NAME);  
12 end;  
13  
14 select * from hist_usuario;
```

Estrutura de uma Trigger

- CREATE OR REPLACE TRIGGER nome_da_trigger
- [BEFORE or AFTER] [INSERT OR UPDATE OR DELETE]
- ON nome_da_tabela
- [FOR EACH ROW] -- se quiser disparar linha a linha
- DECLARE
- -- variáveis
- BEGIN
- -- implementação
- END;

Before e After

- O before quer dizer antes que as alterações sejam feitas nos registros da tabela em questão, ou seja, podemos dizer que o Oracle faz as alterações primeiramente em memória, para depois efetivá-las no banco de dados.
- Com isso, dependendo do momento tratado, before ou after, podemos ou não alterar estes dados.
- Quando o trigger é disparado no momento before, os dados ainda não foram efetivados, neste caso podemos alterar os valores antes que eles sejam gravados no banco de dados. Quando é disparado no after não há mais como alterá-los, pois já foram efetivados no banco de dados.

Tabel comparativa para os tipos

Tipo	Execução	Exemplo de uso
BEFORE INSERT	Antes de Inserir	Validar valores
AFTER INSERT	Depois de Inserir	Log
BEFORE UPDATE	Antes de Alterar	Impedir alteração; Validar valores
AFTER UPDATE	Depois de Alterar	Log
BEFORE DELETE	Antes de Excluir	Impedir exclusão
AFTER DELETE	Depois de Excluir	Log

:new & :old

código	Descrição	Operações DML	Exemplo
:new	Valor após a alteração	insert ou update	:new.nome_coluna
:old	Valor antes da alteração	update ou delete	:old.nome_coluna

Exemplo 1

```
create table tbConta(  
    idConta number GENERATED ALWAYS AS  
    IDENTITY (START WITH 1 INCREMENT BY 1) primary key,  
    numeroConta varchar(10),  
    saldo float  
);
```

```
insert into tbConta values(default,'111',5000);  
insert into tbConta values(default,'112',-100);  
insert into tbConta values(default,'113',200);  
insert into tbConta values(default,'114',15000);  
insert into tbConta values(default,'115',2000);
```

```
select * from tbConta;
```

```
create or replace trigger trg_saldo_insuficiente  
before update on tbConta  
for each row  
declare  
    v_saldo float;  
  
begin  
    if :new.saldo < 0 then  
        raise_application_error(-20001,'Saldo Insuficiente');  
  
    end if; ss  
end;
```

```
update tbConta set saldo = (saldo + 100) where idConta = 1;
```

Exemplo 2

```
CREATE OR REPLACE TRIGGER trg_auditoria_insercao_funcionarios
AFTER INSERT ON funcionarios
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_funcionarios (id_auditoria, id_funcionario, nome, cargo, salario, data_insercao)
    VALUES (auditoria_funcionarios_seq.NEXTVAL, :NEW.id, :NEW.nome, :NEW.cargo, :NEW.salario, SYSDATE);
END;

INSERT INTO funcionarios (id, nome, cargo, salario) VALUES (1, 'Bruna Silva', 'Analista', 3000);
INSERT INTO funcionarios (id, nome, cargo, salario) VALUES (2, 'Carla Santos', 'Gerente', 5000);

-- Verificando a tabela de auditoria
SELECT * FROM auditoria_funcionarios;
```

Exercícios

- Dadas as tabelas:

```
create table tbCliente(  
    idCliente number GENERATED ALWAYS AS IDENTITY primary key,  
    nomeCliente varchar2(50),  
    dateNasc date  
);  
  
create table tbUsuario(  
    idUsuario number GENERATED ALWAYS AS IDENTITY primary key,  
    nomeUsuario varchar2(50),  
    senhaUsuario varchar2(40)  
);  
  
create table tbLogCliente(  
    idLogCli number GENERATED ALWAYS AS IDENTITY primary key,  
    idCliente number,  
    idUsuario number,  
    rotina varchar2(40),  
    dataOperacao DATE DEFAULT SYSDATE  
);
```

1. Crie uma trigger que salve na tabela tbLogCliente os dados ao inserir, alterar ou excluir um registro na tabela de cliente.
2. Crie um trigger que ao alterar ou excluir dados, salve em uma tabela os dados antigos. (Exemplo: crie uma tabela de histórico).

Códigos necessários para a realização do próximo exercício

- `select sysdate from dual;`
- `select trim(to_char(sysdate,'DAY')) "Dia" from dual;`
- `select trim(to_char(sysdate,'DAY','NLS_DATE_LANGUAGE=PORTUGUESE')) "Dia" from dual;`
- `select trim(to_char(sysdate,'MONTH','NLS_DATE_LANGUAGE=PORTUGUESE')) "Mes" from dual;`
- `select trim(to_char(sysdate,'YYYY')) "Ano" from dual;`

Exercício

- Criar uma trigger que exiba uma mensagem de erro ao tentar inserir, editar ou excluir um registro na quinta-feira. Caso contrário o script pode ser executado.

Contato

E-mail: allan@cruzeirosul.edu.br



Universidade **Cruzeiro do Sul**

www.cruzeirodosul.edu.br